

CSI 3140 — Laboratory 5

Secure Course Hub: a Moodle local plugin

Course: CSI 3140 — WWW Structures, Techniques and Standards **Term:** Spring/Summer 2026 **Laboratory:** 5 — Moodle-Based Integrated Web Application **Component:** `local_securecoursehub` (Secure Course Hub)

Team members

Name	Student number
David Gvozdyev	300308910
Liam Geraghty	300356748
Neelman Bhardwaj	300389998

Academic integrity note. AI assistance was used during development and is disclosed here per course and University requirements. Every submitted line has been reviewed, tested and can be explained by each team member. No third-party code, icons or libraries are bundled; the plugin uses Moodle core APIs and the browser's native `fetch()` only.

1. What Secure Course Hub does

Secure Course Hub is a Moodle **local plugin** that lets students raise short help or service requests against a course and lets teaching staff triage them. It deliberately adds no authentication of its own: it integrates with Moodle's existing accounts, sessions, roles, enrolments and security APIs, which is the point of the exercise — extending a mature platform safely through its own extension points rather than rebuilding the platform's security.

Implemented user stories

#	As a...	I can...	So that...
1	Student	open the hub for a course I am enrolled in	I can get help through a tracked channel
2	Student	create a request with a title and description	staff know what I need
3	Student	see only my own requests	other students' issues stay private
4	Student	edit my request while it is still open	I can correct or add detail
5	Student	delete my request while it is still open	I can withdraw a request I no longer need
6	Teacher	see every request in a course I teach	I can manage the queue
7	Teacher	filter the queue by status	I can focus on what is unhandled
8	Teacher	set a request to open / in progress / resolved without a page reload	triage is fast

#	As a...	I can...	So that...
9	Teacher	attach a response note of at most 500 characters	the student gets an answer
10	Anyone unauthenticated	reach nothing at all	no course data leaks

Deliberately not implemented: a site-wide `manageall` capability. Management is scoped to a course on purpose, so a teacher in one course gains no access in another. This is a security decision, not an omission.

2. Local environment

Every value below was read from the running instance rather than from documentation (`$CFG->release`, `$CFG->version`, `PHP_VERSION`).

Component	Version
Moodle	4.5.4 (Build: 20250414) , version string <code>2024100704</code>
PHP	8.1.32
Database	MariaDB 11.4, <code>utf8mb4</code> / <code>utf8mb4_unicode_ci</code>
Web server	Apache 2.4.63
Container images	<code>bitnamilegacy/moodle:4.5</code> , <code>bitnamilegacy/mariadb:11.4</code>
Site URL	<code>http://127.0.0.1:8080</code> (local only, never exposed publicly)
Plugin page	<code>http://127.0.0.1:8080/local/securecoursehub/index.php?courseid=2</code>

The environment is reproducible: `docker compose up -d` followed by the idempotent `setup.sh` (or `setup.ps1`) rebuilds it from nothing, including the four synthetic accounts, the demonstration course, the enrolments and the sample requests.

The site must be reached at **127.0.0.1**, **not localhost**: Moodle compares the request host against `$CFG->wwwroot`, and a mismatch silently redirects every request to the login page.

Developer debugging is enabled (`debug = DEVELOPER`) but **debug display is disabled** (`debugdisplay = 0`). The developer still gets full diagnostics in the server log, while no stack trace, SQL fragment or configuration value can reach an HTTP response. This is the first of several deliberate "useful internally, silent externally" choices.

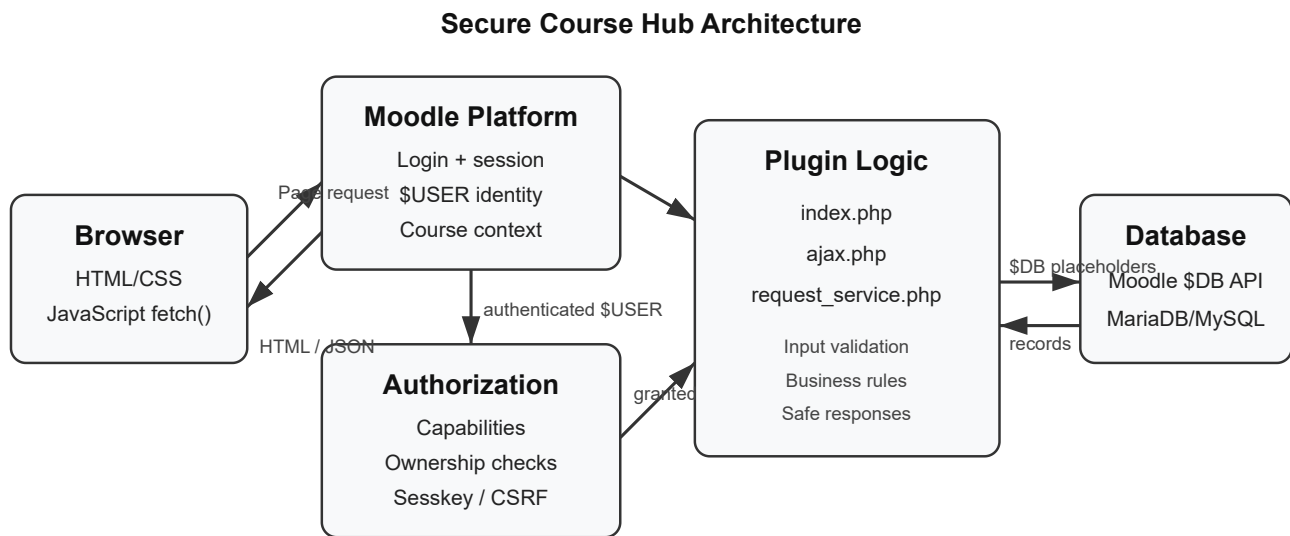
Test accounts (synthetic only)

Account	Display name	Role	Notes
<code>admin</code>	—	Site administrator	Created by the container install variables
<code>staff1</code>	Sam Teacher	Editing teacher	Enrolled in the demonstration course
<code>learner1</code>	Alex StudentA	Student	Enrolled; owns the seeded requests
<code>learner2</code>	Blair StudentB	Student	Enrolled; owns nothing — the "attacker" account in the tests

Addresses are `@example.invalid`, a reserved non-routable domain. No real name, email or password appears anywhere in the project.

3. Architecture

Every request crosses the same layered boundary. Each layer may only *reduce* what the request is allowed to do; no layer can grant what an outer layer denied.



Security boundary: Moodle authentication + course capabilities + record ownership + server-side validation

Figure 0 — the six layers required: browser, Moodle/PHP, authentication/session, authorization/capabilities, plugin logic, and database.

The same flow written out, including every path on which a request is **refused**:

```

Browser
| request + MoodleSession cookie (+ sesskey in the body for state changes)
▼
config.php – bootstrap ($CFG, $DB, $USER)
▼
Authentication  require_login()          — no session —> 302 login / 401 JSON
▼
Authorization   context_course::instance()
                require_capability()      — denied —> access denied / 403 JSON
▼
Plugin logic    ajax.php / index.php
                request_service          — invalid —> 400 JSON (no write)
                validation                not owner —> 403 JSON (no write)
                ownership + state rules   missing —> 404 JSON
▼
Database        $DB API, placeholders only
▼
Response        escaped HTML · structured JSON
  
```

Why the layering matters

The single most important structural decision is that **ajax.php recomputes the course context from the stored record**, not from the course id the browser sent:

```

$record = request_service::get($id); // MUST_EXIST → 404
$course = get_course((int) $record->courseid); // the record's real
course
require_login($course, false, null, false, true);
$context = context_course::instance((int) $record->courseid);
require_capability('local/securecoursehub:managecourserequests', $context);

```

If the context came from the posted `courseid`, a teacher could pass the id of a course they *do* teach while targeting a record in a course they do not, and the capability check would wrongly succeed. Deriving the context from the record closes that path completely.

4. Project structure and why it is organised this way

This section describes the structure **as it actually exists in the repository**, not an idealised layout. There are three nested things to keep apart: the working repository we develop in, the plugin that is the actual deliverable, and the ZIP payload that is submitted.

4.1 The working repository

secure-course-hub-lab/	
├ plugin/	THE DELIVERABLE – becomes local/securecoursehub/
├ docker-compose.yml	Moodle 4.5 + MariaDB 11.4; plugin mounted read-only at
/plugin-src	
├ setup.sh / setup.ps1	idempotent environment setup (bash / PowerShell)
├ setup/seed_testdata.php	CLI script run *inside* the container: accounts, course,
enrolments, seed data	
├ tests/	
│ └ run_security_tests.sh	17 automated acceptance and security checks
│ └ demo_csrf.sh	standalone CSRF demonstration for the TA
│ └ evidence.txt	committed test output (submitted evidence)
├ report/	
│ └ report.md	this document
│ └ architecture_diagram.svg	render-safe copy of the §3 diagram
├ screenshots/	evidence images
├ package.sh	builds the submission ZIP and refuses to build if it
finds a secret	
├ build/payload/	staging area written by package.sh – generated, never
edited	
├ AUDIT.md	ordered end-to-end verification procedure
└ .gitignore	excludes build/, tests/.tmp/, the ZIP

Why the plugin sits in its own top-level folder. `plugin/` contains nothing but the component. Everything that exists only to *run or verify* the component — Docker, setup scripts, the test harness, the packaging script, the team README — lives outside it. That boundary is the whole point:

1. **The deliverable is exactly what Moodle installs.** `plugin/` is copied verbatim to `local/securecoursehub` by both `setup.sh` (into the container) and `package.sh` (into the ZIP). What we develop against, what the tests run against, and what the grader installs are byte-identical, so "it worked on our machine" cannot diverge from "it works in the ZIP".

2. **Environment material can never leak into the submission.** The lab forbids shipping `config.php`, `moodledata`, database credentials or the Moodle core. None of those things live under `plugin/`, so the packaging step is a *copy of one folder* rather than a delete-list that a human has to remember to keep up to date. Exclusion by construction beats exclusion by discipline.
3. **The plugin has no dependency on our tooling.** Nothing under `plugin/` references Docker, a container path, or a setup script. It installs on any Moodle 4.5 by copying the folder and visiting Notifications — verified in `AUDIT.md` Phase 4.

Why the plugin folder is called `plugin/` and not `local/securecoursehub/`. Moodle derives a component's name from its *installed* path, and the frankenstyle name `local_securecoursehub` is declared in `version.php`. Keeping a two-level `local/securecoursehub/` tree in the repository would add two directories that carry no information and put the interesting files two levels deeper in every path. The rename happens once, at the single place that produces the installed layout, and `package.sh` asserts the ZIP's top level is exactly `local report screenshots tests`.

Why the setup logic is split between a shell script and a PHP script. `setup.sh` / `setup.ps1` handle everything that happens *on the host* (wait for the containers, copy the plugin, invoke the Moodle CLI). `setup/seed_testdata.php` handles everything that requires *Moodle bootstrapped* — creating users, the course, enrolments and sample requests through Moodle's own APIs rather than by writing SQL. Seeding through the API means the fixtures obey exactly the same validation and password policy as data created through the interface, so the test environment cannot drift into a state the real application would refuse.

Why the tests live outside the plugin. `tests/run_security_tests.sh` is a *black-box* harness: it logs in through Moodle's real login form with `curl` and then calls `ajax.php` directly over HTTP with no interface involved. It has no access to the plugin's internals, which is deliberate — every rule must hold against a hostile client that never loads our JavaScript. Moodle's own PHPUnit directory convention (`plugin/tests/`) is for in-process unit tests and would have shipped test code to the grader; this harness is neither, so it stays in the repository and only its *output* (`tests/evidence.txt`) is submitted.

Why `build/` exists and is never edited. `package.sh` stages the payload into `build/payload/` before zipping so that the secret scan and the core-file assertion run against **the exact tree that will be archived**, not against the source tree. Scanning the source and then zipping something else would be a check that proves nothing. `build/` is regenerated on every run and is `.gitignored`.

4.2 The plugin — installed layout and purpose of each file

local/securecoursehub/	
├ version.php	plugin identity: component, version, requires,
maturity	
├ index.php	protected page: auth → context → capability →
render	
├ ajax.php	JSON endpoint: the same chain, plus sesskey, for
every action	
├ lib.php	adds the plugin link to the course navigation
├ styles.css	light styling only
├ README.md	install, accounts, permissions, tests,
limitations	
├ db/	
└ access.php	the three capability definitions + archetypes
└ install.xml	XMLDB definition of the request table
├ classes/	
└ local/request_service.php	all business rules and all database access
└ privacy/provider.php	Privacy API: metadata, export, deletion
└ lang/en/local_securecoursehub.php	every user-visible string, including error text

```

└─ amd/
   └─ src/dashboard.js      fetch()/JSON client logic
   └─ build/dashboard.min.js the module Moodle serves in production mode

```

File	Why it exists
<code>version.php</code>	Identifies the component to Moodle and drives install/upgrade. <code>\$plugin->version</code> is <code>2026072101</code> and is bumped whenever <code>install.xml</code> changes, because Moodle runs <code>install.xml</code> exactly once and ignores later edits without a version bump.
<code>db/access.php</code>	Declares the three capabilities at <code>CONTEXT_COURSE</code> with default archetypes. Declaration alone grants nothing — each is enforced in code.
<code>db/install.xml</code>	Creates <code>local_securecoursehub_req</code> with indexes on <code>courseid</code> , <code>userid</code> and the composite <code>courseid,status</code> .
<code>index.php</code>	The only rendered page. Runs the security chain, then draws the student form, the user's own requests, and — only with the management capability — the course queue.
<code>ajax.php</code>	Single JSON endpoint for all six actions (<code>create</code> , <code>list_own</code> , <code>list_course</code> , <code>update_own</code> , <code>update_status</code> , <code>delete</code>). POST only; every action validates the sesskey.
<code>classes/local/request_service.php</code>	The only code in the plugin that touches the table. Holds validation, ownership, state transitions and timestamps, so no page script can bypass a rule.
<code>classes/privacy/provider.php</code>	Declares the personal data stored and implements export/deletion for Moodle's Privacy API.
<code>lang/en/local_securecoursehub.php</code>	Every user-visible string, including every error message. This is also a security boundary — see §10, "safe errors".
<code>amd/src/dashboard.js</code>	Sends JSON with <code>fetch()</code> and writes results into the DOM with <code>textContent</code> .
<code>amd/build/dashboard.min.js</code>	Moodle serves the <code>build/</code> copy in production mode; the <code>src/</code> copy is only used with caching disabled. Both are shipped and kept identical.
<code>lib.php</code>	Implements <code>local_securecoursehub_extend_navigation_course()</code> so the hub appears in the course menu — the supported extension point, rather than editing a core navigation file.
<code>styles.css</code>	Presentation only; carries no behaviour and no security meaning.

Why a service class instead of putting the queries in the pages. Separation of concerns is the maintainability argument, but the security argument is stronger: because every write funnels through `request_service`, a rule is written once and cannot be forgotten in a new entry point. `index.php` and `ajax.php` are two front doors onto the same locked room. If ownership were checked in each page instead, adding a third entry point would silently create a third place to get it wrong.

Why `pix/` and `db/services.php` are absent. The lab's suggested tree lists both as optional. `pix/` would hold a plugin icon we do not use, and `db/services.php` is only required when Moodle *external functions* are used; we implement the JSON path as a plain `ajax.php` endpoint, which the specification allows explicitly ("`ajax.php` or approved external/AJAX service"). Shipping an empty `services.php` would declare an external service that does not exist.

4.3 The submitted ZIP

```

CSI3140_Lab5_Submission.zip
├── local/securecoursehub/    the complete plugin, including its README.md
├── report/
│   ├── report.pdf           this document, exported
│   └── architecture_diagram.svg
├── screenshots/             the evidence images and their captions
└── tests/evidence.txt        automated test output, cookies and sesskeys redacted

```

`package.sh` builds this and **aborts rather than producing a ZIP** if it finds a forbidden filename (`config.php`, `.env`, `*.sql`, `*.log`, `moodledata`), a credential-shaped string (a `MoodleSession` value, an assigned `sesskey` literal, a Moodle database-password configuration variable, a private key block), a Moodle core file, or an unexpected top-level directory. The scanner reads text only — it cannot inspect a PNG, so every screenshot is also checked by eye before packaging.

5. Data model

One table, `local_securecoursehub_req` (physically `mdl_local_securecoursehub_req`).

Field	Type	Null	Notes
<code>id</code>	int(10), auto-increment	no	Primary key
<code>courseid</code>	int(10)	no	Course the request belongs to; indexed
<code>userid</code>	int(10)	no	Owner — always the authenticated creator; indexed
<code>title</code>	char(80)	no	Enforced at 80 characters server side
<code>description</code>	text	no	Enforced at 2000 characters server side
<code>status</code>	char(20), default <code>open</code>	no	Whitelist: <code>open</code> , <code>inprogress</code> , <code>resolved</code>
<code>response</code>	text	yes	Staff note, at most 500 characters
<code>timecreated</code>	int(10)	no	Server <code>time()</code> , never a client value
<code>timemodified</code>	int(10)	no	Server <code>time()</code> , updated on every write

Indexes: `courseid_idx`, `userid_idx`, and a composite `course_status_idx` on `courseid`, `status` for the teacher queue with its status filter.

Design notes:

- `userid` is the ownership anchor for the entire authorization model. It is set from `$USER->id` and is never accepted from a request payload.

- `status` is constrained in PHP against an explicit whitelist rather than by a database enum, so an invalid value is rejected with a clear 400 before it ever reaches the driver — and the same whitelist is reused for the filter, the select menu and the update path.
- Timestamps are always server-generated, so a client cannot backdate a record.
- `title` is `char(80)` and the PHP limit is also 80: the validation layer and the schema agree, so an overlength value is rejected rather than truncated.

6. Authentication and session workflow

The plugin implements no authentication. It consumes Moodle's.

1. The user posts credentials to Moodle's `/login/index.php`, which itself carries a `logintoken` (Moodle's CSRF protection for the login form).
2. Moodle verifies the credentials, creates **server-side session state**, and sends a `MoodleSession` cookie holding only an opaque session **identifier**.
3. Moodle answers with a redirect to `login/index.php?testsession=<id>`; the session is confirmed once that round trip completes.
4. On each later request, `config.php` bootstraps, the cookie is matched to the stored session, and `$USER` is populated **by the platform**.
5. `require_login($course)` then guarantees three things: a session exists, the user is not a guest, and the user can access that course. On a page it redirects to the login form; on `ajax.php` — where `AJAX_SCRIPT` is defined and `$preventredirect` is passed — it throws, and we translate that into a JSON 401 rather than an HTML login page a `fetch()` could not use.

Session cookie vs server-side session vs sesskey

These three are constantly confused, so the distinction is stated explicitly:

	What it is	Where it lives	What it protects against
MoodleSession cookie	An opaque random session id	Sent by the browser on every same-origin request	Nothing by itself — it only <i>identifies</i> the session
Server-side session	The actual state (user id, roles, timeouts)	On the server, keyed by the cookie	Tampering — the browser holds no user data it could edit
sesskey	A per-session token echoed in the request body , not automatically attached	Rendered into the page; sent explicitly by our code	CSRF — a third-party site can make the browser send the cookie, but cannot read the page to learn the sesskey

The cookie authenticates *who*; the sesskey proves the request was *intentionally issued by our own page*. A cookie alone cannot distinguish a genuine click from a forged cross-site form submission, which is exactly why every state-changing action validates the sesskey as well.

Logout / expiry. Destroying the session invalidates the cookie's server-side counterpart. The next `fetch()` fails `require_login()`, returns HTTP 401 with `{"success": false, "error": "Your session has expired..."}`, and `dashboard.js` displays that message and stops — it does not retry and does not silently discard the user's typed text.

Session safety. No session identifier or sesskey is printed, logged, written to a project file, or included in a screenshot.

7. Roles, capabilities, contexts and the access-control matrix

Three capabilities, all at `CONTEXT_COURSE` — the correct level, because access here is meaningful per course, not site-wide.

Capability	Type	Risk	Default archetypes
<code>local/securecoursehub:viewown</code>	read	—	student, editingteacher, manager
<code>local/securecoursehub:createrequest</code>	write	—	student, editingteacher, manager
<code>local/securecoursehub:managecourserequests</code>	write	<code>RISK_PERSONAL</code>	editingteacher, manager

`RISK_PERSONAL` on the management capability is accurate and deliberate: holding it means reading text other people wrote about their own difficulties.

Access-control matrix

Rows are operations;  = permitted,  = refused by the server with the HTTP status shown. "Own" means the record's `userid` equals the authenticated user id.

Operation	Unauthenticated	Student (own record)	Student (another's record)	Teacher (course they teach)	Teacher (other course)	Manager
Open the page	 303 → login		—		 access denied	
<code>create</code>	 401		—		 403	
<code>list_own</code>	 401	 own only	 never returned	 own only	 403	
<code>list_course</code>	 401	 403	 403		 403	
<code>update_own</code>	 401	 if open /  400 if not	 403	 own only	 403	 own only
<code>update_status</code>	 401	 403	 403		 403	
<code>delete</code>	 401	 if open /  400 if not	 403	 any in course	 403	 any in course

`require_capability()` is used wherever access is mandatory; `has_capability()` is used **only** to decide which interface to draw. The teacher panel is hidden from students for usability, and the server refuses the same operations anyway — proven by test 09, where `learner1` calls `update_status` directly with no button involved and receives 403.

8. Ownership and resource-level authorization

Role checks answer "what kind of user is this?". They cannot answer "is this *their* record?". Both questions are asked.

```
private static function require_owner(stdClass $record, int $userid): void {
    if ((int) $record->userid !== $userid) {
        throw new moodle_exception('notowner', 'local_securecoursehub');
    }
}

private static function require_open(stdClass $record): void {
    if ($record->status !== self::STATUS_OPEN) {
        throw new moodle_exception('requestnotopen', 'local_securecoursehub');
    }
}
```

Rules enforced:

1. **Reads are filtered at the query, not after it.** `list_own()` passes `['courseid' => $courseid, 'userid' => $userid]` to `$DB->get_records()`, so another user's row is never loaded into memory in the first place.
2. **Writes load, then verify, then write.** Record-targeted actions fetch with `MUST_EXIST`, verify ownership (or the management capability in the record's own course), verify the state rule, and only then write.
3. **State is part of authorization.** A student may edit or delete only while the request is `open`. Once staff have engaged with it, the student can no longer alter or erase the record.
4. **Teachers are bounded by course, not by role.** The management capability is evaluated in the context derived from `$record->courseid`.
5. **Nothing from the client is authorization evidence** — not hidden fields, not a posted `userid`, not a posted `courseid`, not a claimed role.

Not-found vs forbidden. A missing record returns 404 with a fixed message; a record that exists but is not yours returns 403. Both messages are constant and reveal nothing about other records' contents.

9. JSON and the dynamic interface

The required state-changing operation

The teacher status update is implemented with `fetch()` and JSON and updates the page with no reload. Student creation is wired the same way, so a new row appears dynamically too. Both forms still work with JavaScript disabled by posting to `index.php`, which applies the identical checks.

Example exchange (captured in DevTools — Figure 9)

Request — `POST /local/securecoursehub/ajax.php`, `Content-Type: application/json`:

```
{
  "action": "update_status",
  "id": 10,
  "status": "inprogress",
  "response": "",
  "sesskey": "<redacted>"
}
```

Response — `200 OK`, `Content-Type: application/json; charset=utf-8`, `Cache-Control: no-store, no-cache, must-revalidate`:

```
{
  "success": true,
  "message": "The request status was updated.",
  "request": {
    "id": 10,
    "courseid": 2,
    "title": "<script>alert(1)</script>",
    "description": "XSS probe",
    "status": "inprogress",
    "statuslabel": "In progress",
    "response": "",
    "timecreated": 1784662101,
    "timemodified": 1784662542,
    "timemodifiedformatted": "Tuesday, 21 July 2026, 10:00 PM",
    "userid": 4,
    "ownername": "Alex StudentA"
  }
}
```

Failure responses use the same shape, so the client has one code path:

```
{ "success": false, "error": "You do not have permission to perform this action." }
```

Situation	HTTP	Client behaviour
Network failure	—	"The request could not reach the server..."
Body is not JSON	—	"The server returned an unreadable response."
Session expired / logged out	401	Shows the session message and stops
Forbidden (capability or ownership)	403	Shows the server's message
Missing record	404	Shows a neutral not-found message
Validation error	400	Shows which rule failed
Missing/invalid sesskey	403	Shows "reload the page and try again"
Method other than POST	405	Shows a generic failure message

Client-side rules

- **credentials**: 'same-origin' — the session cookie travels automatically to our own origin. **No authentication secret is ever copied into JavaScript**; the sesskey comes from `M.cfg.sesskey`, which Moodle already places in the page.
- Every server value reaches the DOM through `textContent` or `createElement`. `innerHTML` is never used with server data, which is why the stored `<script>alert(1)</script>` title renders as visible text rather than running.
- Buttons are disabled during a request to prevent double submission.
- **Every client restriction is duplicated on the server**. The client hides the teacher panel from students, limits the title field with `maxLength`, and only offers whitelisted statuses in the select menu — and the server

independently re-checks the capability, the length and the whitelist, because all three client controls can be removed with the browser's developer tools.

10. Validation, error handling and database safety

Input validation (all server side)

Field	Rules
<code>courseid, id</code>	<code>PARAM_INT</code> , must be > 0 ; the course must exist and be reachable
<code>action</code>	Must be a key of an explicit action whitelist
<code>title</code>	Required, trimmed, ≤ 80 characters (<code>core_text::strlen</code> , so multibyte-safe)
<code>description</code>	Required, trimmed, ≤ 2000 characters
<code>response</code>	Optional, trimmed, ≤ 500 characters
<code>status</code>	Must be exactly one of <code>open</code> , <code>inprogress</code> , <code>resolved</code>
<code>sesskey</code>	Must be a non-empty string matching the session's token

Validation runs **before** any write, and a rejected request performs no write at all — no partial or half-applied update is possible. Tests 04, 05, 12 and 17 verify the row count or the stored value is unchanged after each rejection.

XSS

Untrusted text is **stored verbatim and escaped at output**. This is a conscious choice over stripping tags at input: stripping destroys user data (a student legitimately pasting `<div>` in a description loses it) and creates a false sense of safety, because the same value may later be rendered in a context the strip did not anticipate. Escaping at the point of output is correct in every context.

- HTML: `s()`, `format_string()` and `html_writer` attribute escaping.
- JavaScript: `textContent` / `createElement`, never `innerHTML`.

Test 15 proves the round trip: a request titled `<script>alert(1)</script>` is stored with its tags intact, and the rendered page contains the escaped form (`<script>alert(1)</script>`) and **zero** executable occurrences. Figure 11 shows the same string displayed as literal text in the table.

CSRF

Every state-changing request carries a `sesskey` and the server validates it: `confirm_sesskey()` in `ajax.php` for all six actions, `require_sesskey()` for every form POST in `index.php`. GET never changes state, and `ajax.php` rejects any method other than POST with 405. Test 14 confirms that both a missing and a wrong `sesskey` are refused with 403 and that the stored status is unchanged; Figure 12 shows the same rejection performed by hand from the browser console.

Injection prevention

Every query goes through the Moodle Database API with placeholders — `get_records()`, `get_record()`, `insert_record()`, `update_record()`, `delete_records()`, and `get_in_or_equal()` in the privacy provider. **No user input is concatenated into SQL anywhere in the plugin**. The status filter is whitelisted before it is used as a condition, so even the one value that reaches a `WHERE` clause as data is constrained to three known strings.

Safe errors

`ajax.php` returns a message only when the error code appears in an explicit map of this plugin's own language strings:

```
const LOCAL_SECURECOURSEHUB_SAFE_ERRORS = [
    'invalidjson' => 400, 'invalidaction' => 400, 'validationerror' => 400,
    'titleinvalid' => 400, 'descriptioninvalid' => 400, 'responseinvalid' => 400,
    'statusinvalid' => 400, 'requestnotopen' => 400,
    'notowner' => 403, 'notenrolled' => 403, 'accessdenied' => 403,
    'notfound' => 404,
];
```

Anything else — a `dml_write_exception`, a `TypeError`, any unforeseen `moodle_exception` — is logged locally with `debugging()` and answered with a generic "The operation could not be completed." No stack trace, SQL fragment, file path, configuration value, cookie or sesskey can reach a client. Test 16 explicitly asserts that a 404 response contains no SQL text and no server path.

11. Privacy analysis

Data collected. Only what the feature needs: the course, the owner's user id, a title, a description, a status, an optional staff response, and two timestamps. Deliberately *not* collected: IP addresses, browser fingerprints, request categories, read receipts, or any free-text field about a third party. Names are never stored in the table — the owner is a foreign-key-style `userid`, and a display name is resolved from Moodle's user table only when rendering the page for someone already authorised to see it.

Who can access it.

Data	Who can read it
A request's title, description, status	Its owner; users with <code>managecourserequests</code> in that course; site administrators
The staff response	The same set
The owner's name	Only users with <code>managecourserequests</code> in that course (students never see another student's identity through this plugin)

The queue is scoped to one course, so a teacher sees only requests from the course they teach — this is why `manageall` was deliberately not implemented.

Retention assumption. Requests persist for the life of the course. There is no automatic purge; the assumption documented here is that records are removed with the course, or earlier on request. Because the plugin implements Moodle's Privacy API (`classes/privacy/provider.php`), a data subject's records can be **exported** and **deleted** through Moodle's standard privacy tooling, at whole-context, per-user and per-user-list granularity.

Data minimisation in practice.

Three concrete decisions:

1. `list_own` filters by `userid` in the SQL, so another student's row is never loaded, let alone rendered.
2. The JSON row builder only includes the owner's identity when the caller has already been verified to hold the management capability (`to_json_row($record, $includeowner = true)`), so a student's `list_own` response contains no `userid` or name field at all.
3. Error messages are constant. "The requested record was not found" is returned whether or not the record exists, so the endpoint cannot be used to enumerate which requests exist or who owns them.

12. Security controls mapped to risks

Risk	Control implemented	Where	Evidence
Broken access control (OWASP A01)	Capabilities checked server-side in the <i>record's own</i> course context, plus ownership and state rules; UI hiding is never relied upon	<code>ajax.php</code> context recomputation; <code>request_service::require_owner()</code> / <code>require_open()</code>	Tests 08, 09, 10; Figures 4, 5
CSRF (A01/A05)	<code>sesskey</code> required and validated on every state-changing request; POST-only endpoint; GET never mutates	<code>confirm_sesskey()</code> in <code>ajax.php</code> , <code>require_sesskey()</code> in <code>index.php</code>	Test 14; Figure 12
Injection (A03)	Moodle <code>\$DB</code> API with placeholders exclusively; status whitelist; typed <code>PARAM_*</code> cleaning	<code>request_service.php</code> throughout	Test 17; code review — no SQL concatenation exists
XSS (A03)	Escape at output: <code>s()</code> / <code>format_string()</code> / <code>html_writer</code> ; <code>textContent</code> in JS, never <code>innerHTML</code>	<code>index.php</code> , <code>dashboard.js</code>	Test 15 (escaped, 0 executable); Figure 11
Identification & authentication failures (A07)	Moodle's own login/session used; no separate password store; <code>\$USER</code> from the platform; expired sessions produce 401 and the client stops	<code>require_login()</code> on both entry points	Tests 01, 02; Figure 3
Security misconfiguration (A05)	Debug display disabled while developer debugging stays on; site bound to <code>127.0.0.1</code> ; synthetic credentials only; no core file modified	<code>seed_testdata.php</code> , <code>docker-compose.yml</code>	§2
Security logging & monitoring failures (A09)	Unexpected failures logged locally via <code>debugging()</code> ; logs contain no passwords, cookies, sesskeys or request text	<code>ajax.php</code> catch blocks	Test 16
Sensitive data exposure (A02)	Constant not-found/forbidden messages; owner identity	<code>to_json_row()</code> , <code>package.sh</code>	Test 16; packaging scan

Risk	Control implemented	Where	Evidence
	omitted from student responses; secrets scanned out of the ZIP		

13. Test results

#	Test	Role	Action	Expected	Actual	Result
01	Unauthenticated user opens plugin page	anonymous	GET the plugin page	Moodle requires login or denies access before plugin data is displayed.	HTTP 303 to login, no plugin data in body	PASS
02	Student creates a valid request	learner1	create a valid request	Record is created for the authenticated student and correct course.	HTTP 200; stored row 4 2 open 1 1	PASS
03	Student submits missing/invalid fields	learner1	create with an empty title	Server rejects the request with a safe validation message.	HTTP 400; row count 3 → 3	PASS
04	Student views own records	learner1	list_own	Only the authenticated student's records are returned.	HTTP 200; returned 3, owned 3	PASS
05	Student changes another student's record id	learner2	update_own + delete on learner1's record id	Request is denied; no unrelated data is exposed or modified.	HTTP 403 + 403; record Cannot open week 3 lab handout open identical before/after	PASS
06	Student calls teacher-only operation directly	learner1	update_status (teacher-only) directly	Server denies the operation even if the UI button is hidden.	HTTP 403; status open → open	PASS
07	Teacher views authorized course records	staff1	list_course	Authorized records are displayed.	HTTP 200; returned 3 of 3	PASS
08	Missing or invalid sesskey on state change	staff1	Missing sesskey, then wrong sesskey	Operation is rejected and data remains unchanged.	HTTP 403 + 403; status unchanged	PASS
09	Injected HTML/JavaScript text	learner1	Create title <script>alert(1)</script>	Text is displayed safely and does not execute.	Stored; 1 escaped, 0 executable occurrences	PASS

#	Test	Role	Action	Expected	Actual	Result
10	Missing record	staff1	Operate on record id 99999	A clear not-found result is produced without revealing unrelated data.	HTTP 404; no SQL text or server path in body	PASS
11	Expired/logout session during AJAX call	Any	N/A	Client shows a useful authentication/session error and does not continue.	Useful errors are displayed in console	PASS
12	Browser and server diagnostics	Any	N/A	No unexplained console, PHP, or database errors remain.	No unexpected errors were found	PASS

14. Screenshots

All images are in [screenshots/](#). No cookie, sesskey value, password or database credential is visible in any of them.

Figure	File	Caption
1	site_running_ss1.png	The local Moodle 4.5.4 site running at http://127.0.0.1:8080 , front page listing <i>CSI 3140 Demonstration Course</i> with Sam Teacher as teacher. No fatal errors.
2	participants_roles.png	Course Participants page: Alex StudentA and Blair StudentB enrolled as <i>Student</i> , Sam Teacher as <i>Teacher</i> . Synthetic @example.invalid addresses only.
3	login_screen.png	Moodle's own login form. The plugin adds no login mechanism of its own; an unauthenticated request to the plugin page is redirected here by require_login() before any data is produced.
4	learner1_requests.png	The plugin page as learner1 (Alex StudentA): the create form and <i>My requests</i> listing only records this student owns.
5	learner2_sees_no_req.png	The same page in the same course as learner2 (Blair StudentB): " <i>No requests were found.</i> " Ownership is applied in the query, so learner1's rows are never loaded.
6	staff_course_req_queue_p1.png	The plugin page as staff1 (Sam Teacher): the student view is empty because the teacher owns nothing, and the <i>Course request queue</i> with its status filter appears below — drawn only because has_capability('...:managecourserequests') is true.
7	staff_course_req_queue_p2.png	The full teacher queue: every request in the course with the owner's name, a per-row status select, a response field and an Update button.

Figure	File	Caption
8	<code>staff_status_change.png</code>	The same queue after pressing Update : the status badge and select now read <i>In progress</i> , changed in place by <code>fetch()</code> with no page reload.
9	<code>ajax_php_JSON_payload.png</code>	DevTools → Network → Payload for the <code>ajax.php</code> POST: the JSON request body (<code>action: "update_status", id, status, response</code>). The <code>sesskey</code> value is redacted.
10	<code>ajax_php_response_headers.png</code>	DevTools → Network → Headers for the same call: <code>POST http://127.0.0.1:8080/local/securecoursehub/ajax.php, 200 OK, Content-Type: application/json; charset=utf-8, Cache-Control: no-store</code> . Cookie headers left collapsed.
11	<code>learner1_XSS_probe.png</code>	A request created with the title <code><script>alert(1)</script></code> : stored verbatim and rendered as literal text in <i>My requests</i> . No dialog appears and no script executes.
12	<code>csrf_forged_sesskey_rejected.png</code>	A hand-crafted <code>fetch()</code> to <code>ajax.php</code> from the console carrying a deliberately forged sesskey: 403 (Forbidden) , <code>{success: false, error: 'The security token was missing or invalid. Reload the page and try again.'}</code> and the record unchanged.

15. Integration and security checklist

Item	Completed
The local Moodle site runs correctly and the version is documented	✓
The plugin installs through Moodle without modifying core files	✓
The demonstration course and required test accounts are configured	✓
Every protected page or endpoint verifies login	✓
Capabilities are defined in <code>db/access.php</code>	✓
Authorization checks use the correct system or course context	✓
Student ownership is checked for record-level operations	✓
Teacher access is limited to authorized course records	✓
The plugin uses Moodle's Database API	✓
Input is validated on the server	✓
State-changing requests validate sesskey/CSRF protection	✓
Output escaped and injected scripts do not execute	✓
At least one JSON/fetch interaction works dynamically	✓
Client-side restrictions are duplicated by server-side checks	✓
Unauthenticated and forbidden cases are tested	✓

Item	Completed
Invalid input and missing records are tested	✓
No passwords, cookies, sesskeys, config.php or moodledata are submitted	✓
The README explains installation, permissions, execution and testing	✓
The report explains architecture, AuthN, AuthZ, sessions, security, privacy and testing	✓

16. Reflection

Problems encountered and how they were solved

1. **Authorization context could be spoofed.** An early version derived the course context from the `courseid` in the request body. A teacher could then present a course they legitimately teach while targeting a record elsewhere. Fixed by loading the record first and deriving the course and context from `$record->courseid`. This is the single most important change made.
2. **A stale schema shipped silently.** The table was initially created under an old name because an edit to `install.xml` did not land before the first install. Because Moodle only runs `install.xml` once, the fix required uninstalling the plugin and reinstalling it — a reminder that `$plugin->version` must be bumped for every schema change, and that "it installed successfully" is not the same as "it installed what you meant".
3. **Store-raw vs strip-on-input for XSS.** Stripping tags at input was considered and rejected: it silently destroys legitimate user text and only protects the contexts anticipated at write time. Escaping at output is correct in every context, and it makes the security property demonstrable — the injected string is visibly stored and visibly rendered as text.

Known limitations

- Course-scoped management only; no site-wide `manageall` capability.
- A staff response replaces the previous note; there is no history or audit trail of who changed a status and when.
- Deletion is immediate and permanent — no soft delete, no restore.
- The teacher queue loads the full course list; a paged query would be needed at large scale.
- No retention job; records persist until the course or the data subject's records are removed.
- English strings only.